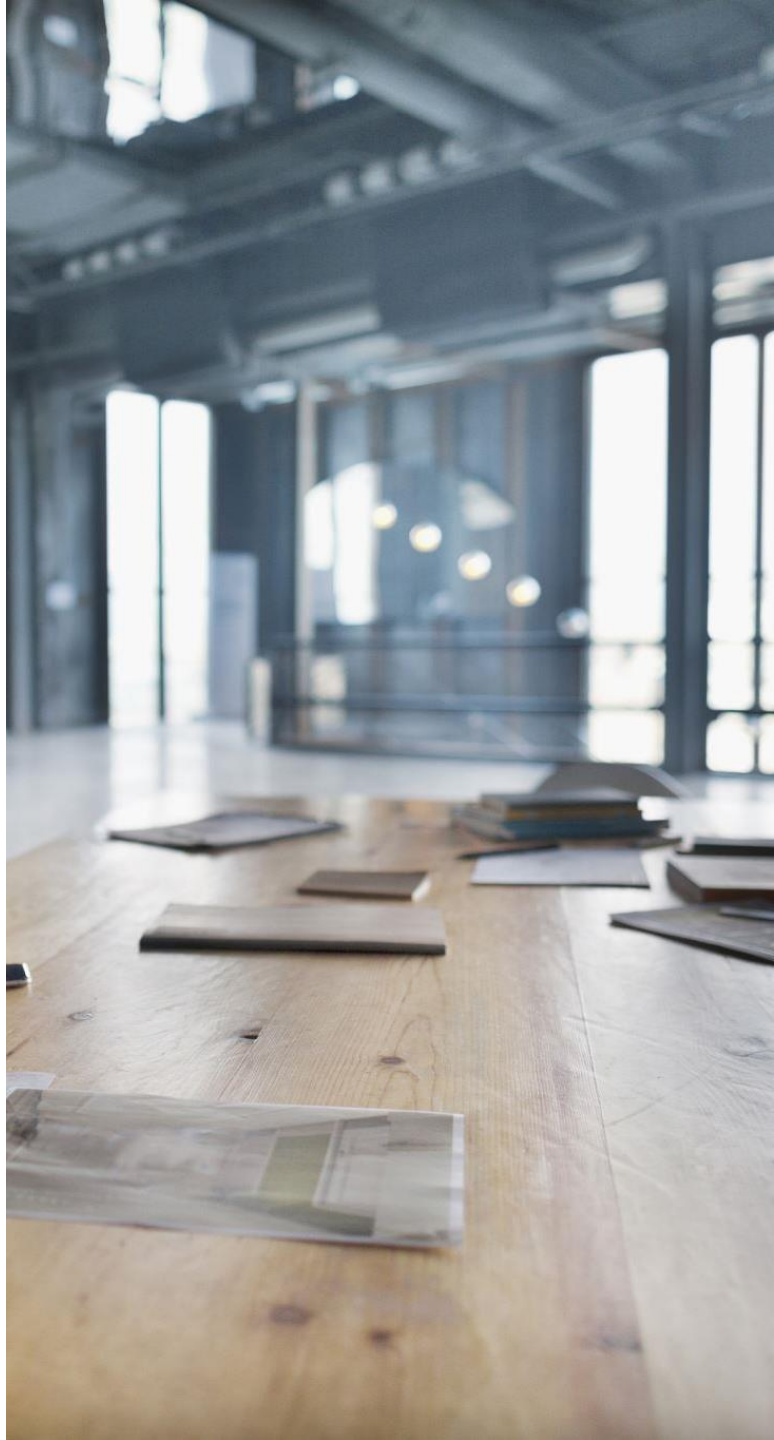
# TRIGGERS

# What is trigger:

In SQL, a trigger is a special stored procedure that runs automatically when a specific event occurs on a table.

- These events include INSERT, UPDATE, and DELETE operations.
- Triggers are mainly used to automate tasks, enforce complex business rules, and maintain data integrity or audit trails inside the database



# Why Use Triggers?

**Content:** Triggers are excellent for:

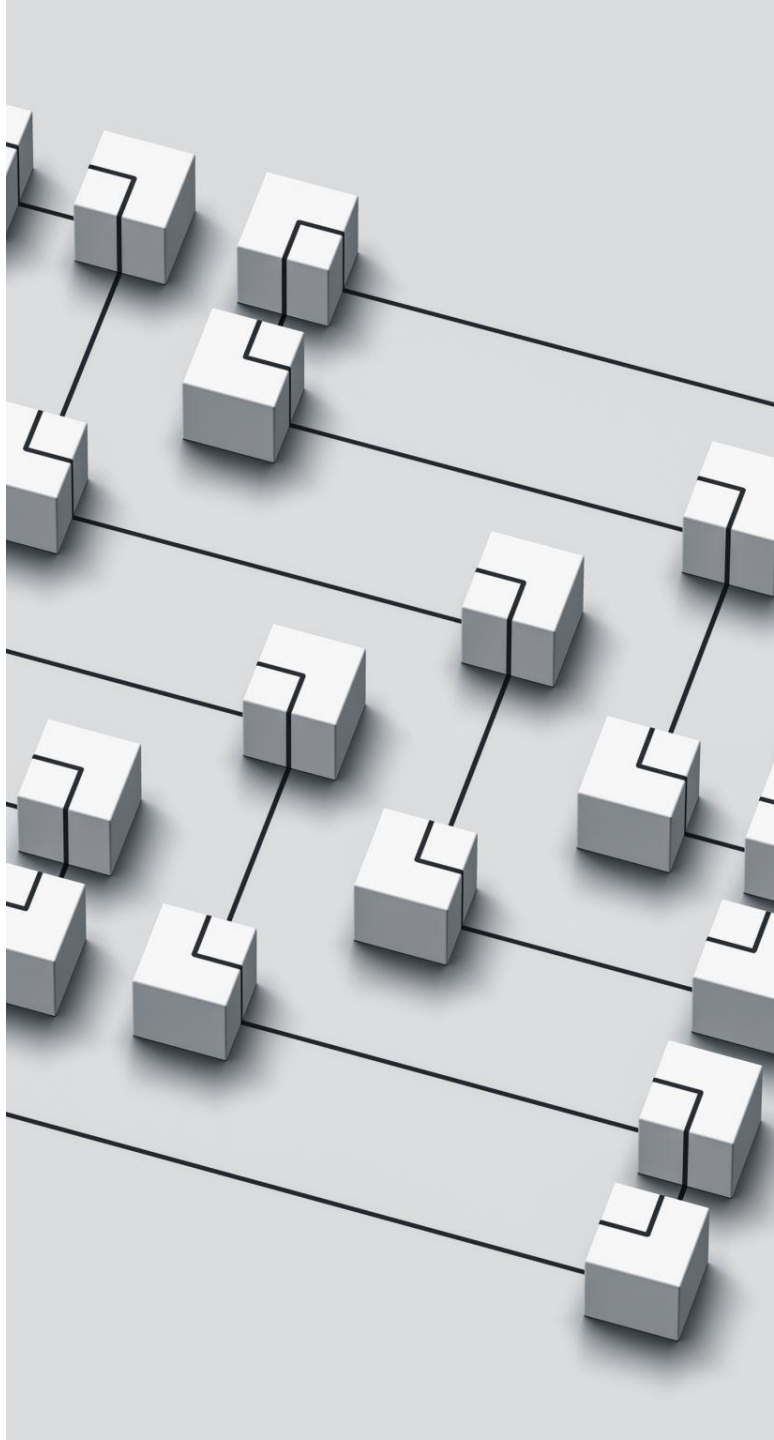
**Auditing:** Transparently logging data modifications (who changed what and when).

**Complex Business Rules:** Enforcing rules like limiting DML operations to specific times/days.

**Data Validation:** Performing cross-table validation before an insert or update.

**Maintaining Replicate Tables:** Keeping data synchronized across tables.

**Security Authorizations:** Implementing complex security checks

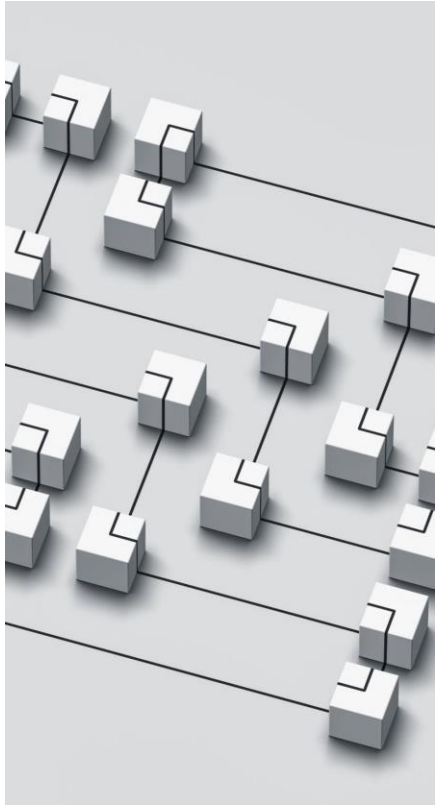


# Types of Triggers (DML, DDL, Logon)

- **Content:** SQL Server supports three main types:
  - **DML Triggers:** Fire in response to `INSERT`, `UPDATE`, and `DELETE` statements on a table or view.
  - **DDL Triggers:** Fire in response to data definition language events like `CREATE`, `ALTER`, `DROP` statements.
  - **Logon Triggers:** Fire when a user session is established with the SQL Server instance.
-

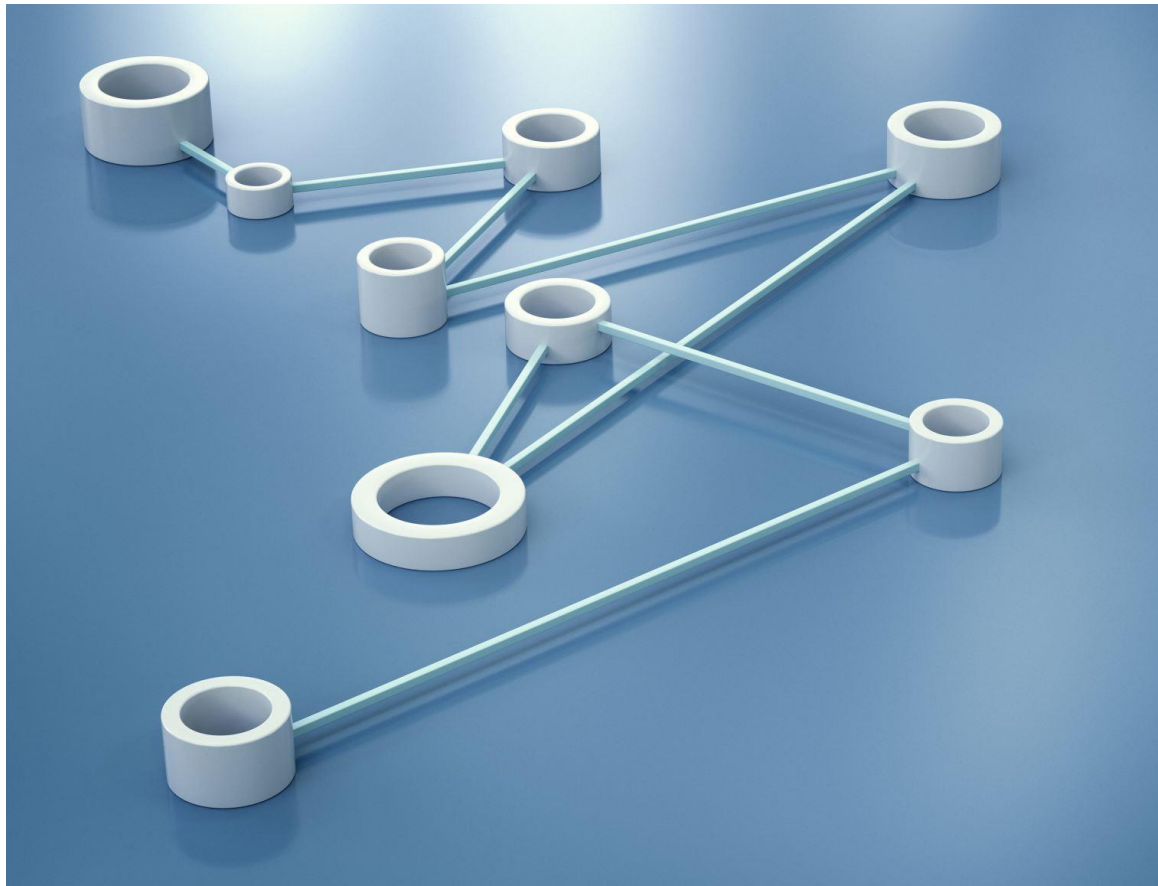
# 1. DDL Triggers

- Data Definition Language (DDL) triggers run when commands like CREATE, ALTER, or DROP are used. They help track or stop changes to the database structure, such as creating, modifying, or deleting tables.
- **Example:** Prevent Table Deletions
- 
```
CREATE TRIGGER prevent_table_creation
ON DATABASE
FOR CREATE_TABLE, ALTER_TABLE, DROP_TABLE
AS
BEGIN
   PRINT 'you can not create, drop and
alter table in this database';
   ROLLBACK;
END;
```



# 2. DML Triggers

- DML triggers fire when we manipulate data with commands like INSERT, UPDATE, or DELETE. These triggers are perfect for scenarios where we need to validate data before it is inserted, log changes to a table, or cascade updates across related tables.
- **Example:** To prevent unauthorized updates in a sensitive students table, we can create a trigger that blocks such changes.
- 
```
CREATE TRIGGER prevent_update
ON students
FOR UPDATE, INSERT, DELETE
AS
BEGIN
    RAISERROR ('You can not insert, update and
delete rows in this table.', 16, 1);
END;
```

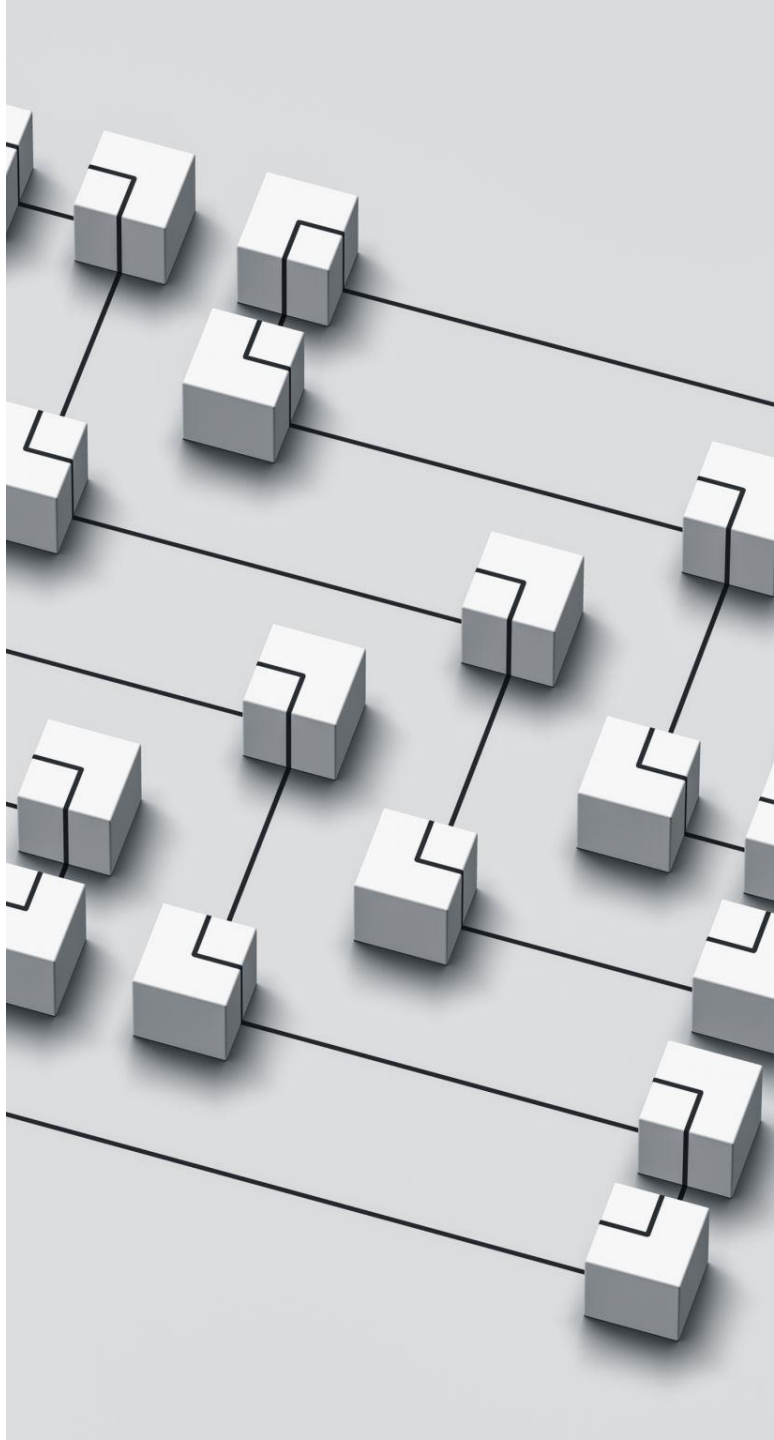


# 3. Logon Triggers

Logon triggers run when a user logs in. They are used to track logins, control access, and limit sessions. Messages from these triggers are stored in the SQL Server error log.

**Example:** Track User Logins

```
CREATE TRIGGER track_logon
ON LOGON
AS
BEGIN
   PRINT 'A new user has logged in.';
END;
```



It supports three main events that can activate a trigger:

- INSERT – When a new row is added to a table.
- UPDATE – When an existing row is modified.
- DELETE – When a row is removed from a table.

Trigger Timing (When It Executes)

- Triggers can execute either BEFORE or AFTER the event.
- BEFORE Trigger: Executes before the actual operation happens.
- AFTER Trigger: Executes after the operation is completed

- **Trigger Scope: Row-Level vs. Statement-Level**
- **Title:** Trigger Scope: Row vs. Statement
- **Content:** Determines how many times a trigger fires.
  - `FOR EACH ROW` **(Row-Level):** Fires once for each row affected by the DML statement. This is the most common type.
  - **Statement-Level:** Fires only once per triggering statement, regardless of the number of rows affected.

# General Syntax of a Trigger

- CREATE TRIGGER trigger_name
- BEFORE | AFTER INSERT | UPDATE | DELETE
- ON table_name FOR EACH ROW
- BEGIN
- •SQL statements;
- END;

## - Example 1: BEFORE INSERT Trigger

- • This example automatically sets the created date before inserting data.
- • CREATE TRIGGER before_insert_student
- • BEFORE INSERT ON students
- • FOR EACH ROW
- • BEGIN
- • SET NEW.created_at = NOW();
- • END;

- Example 2: AFTER UPDATE Trigger (Audit Log)
- • This trigger stores old and new salary values after updating employee salary.
- • CREATE TRIGGER after_update_salary
- • AFTER UPDATE ON employees
- • FOR EACH ROW
- • BEGIN
- • INSERT INTO salary_log(emp_id, old_salary, new_salary)
- • VALUES (OLD.id, OLD.salary, NEW.salary);
- END;

# How to Remove a Trigger

If a trigger is no longer required, it can be removed using:

- DROP TRIGGER trigger_name;

Advantages of Using Triggers

- Triggers execute automatically without user intervention.
- They improve data integrity and enforce rules at the database level.
- They are very useful for logging and auditing purposes.

# Limitations:

- Triggers can be difficult to debug.
- Overusing triggers may affect database performance.
- Triggers work only with DML operations (INSERT, UPDATE, DELETE)

# Summary:

Triggers are automatic database programs in MySQL

- They execute when INSERT, UPDATE, or DELETE events occur.
- They help automate tasks, enforce rules, and maintain data consistency.

# THANK YOU